

Best Restaurants in Pennsylvania: Sentiment Analysis and Ratings-Based Ranking

CMPSC 445 Spring 2026

John Teetz

William Lawther

1. Description of the Project

The goal of this project was to figure out which restaurants in Pennsylvania deserve to be called the best, and to do it in a way that goes beyond a simple star rating. Anyone who has looked at restaurant reviews online knows that a 3.5-star average can mean very different things depending on what people are actually saying. Our application combines numerical ratings with natural language sentiment analysis to produce a more complete picture of restaurant quality.

We built a web-based tool that takes restaurant reviews as input and predicts whether the sentiment behind each review is positive or negative. Those predictions are then combined with star ratings using a weighted scoring formula to rank restaurants across Pennsylvania. The final output is a ranked list that reflects not just how high a restaurant scored numerically, but how positively customers actually wrote about their experience.

Users can also type in their own review text and get an instant sentiment prediction, which gives the application a practical, real-time inference capability on top of the ranking system.

2. Significance of the Project

Restaurant ratings on their own are a blunt instrument. A place with a lot of 3-star reviews might have polarizing food that people feel strongly about in both directions, while another place with the same average might be quietly reliable without inspiring much enthusiasm either way. By layering sentiment analysis over the raw ratings, we try to capture some of that nuance.

This matters practically for people trying to find a good meal in Pennsylvania. It also demonstrates something that applies well beyond restaurants: in many real-world domains, the language people use to describe an experience tells you things that a numerical score alone cannot. This project is a small example of how that kind of text signal can be extracted and put to use.

From a machine learning perspective, the combination of TF-IDF text features with logistic regression is a well-established pipeline for sentiment classification, and applying it in the context of a local ranking problem gives it a meaningful, interpretable purpose. The project is

novel in the sense that it is built specifically around Pennsylvania restaurant data and designed to produce actionable rankings rather than just classification labels.

3. Data Collection

The processed dataset contains 1,000 reviews spread evenly across 10 Pennsylvania restaurant names, with each restaurant receiving 100 reviews. The three columns in the final CSV are `restaurant_name`, `review_text`, and `rating`. The rating column contains integer values of either 1 or 5, there are exactly 500 reviews at each rating level, which means the dataset is perfectly balanced between positive and negative examples. This balance is actually helpful for training because it means the model does not need to account for class imbalance.

A sample of the data looks like this:

<code>restaurant_name</code>	<code>review_text</code>	<code>rating</code>
The Corner Bistro	Wow... Loved this place.	5
Maple & Vine	Crust is not good.	1
Riverside Grill	Not tasty and the texture was just nasty.	1
Harbor House	The selection on the menu was great and so were the prices.	5

The review texts range from brief one-liners to more detailed multi-sentence observations. The metadata includes only the three columns above; there is no geographic location data, timestamp, or user identifier included in the dataset.

4. Data Processing and Feature Engineering

Before any features could be extracted, the review text needed to be cleaned. The preprocessing steps applied to each review were: converting all text to lowercase, removing punctuation, and stripping out any characters that were not letters or whitespace. This normalization reduces noise and ensures that words like "Great" and "great" are treated as the same token.

Sentiment labels were created by thresholding on the rating column: any review with a rating of 4 or higher was assigned a positive label (1), and any review with a rating below 4 was assigned a negative label (0). Since the dataset only contains ratings of 1 and 5, this threshold cleanly separates all negative reviews from all positive ones.

For feature engineering, we used TF-IDF (Term Frequency-Inverse Document Frequency) vectorization on the cleaned review text. TF-IDF assigns higher weights to words that appear frequently in a specific review but are relatively rare across the entire corpus, which is exactly the kind of signal that distinguishes a glowing review from a complaint. The vectorizer was fit only on the training set and then applied to transform the test set, which prevents any information leakage from the test data into the feature representation.

No additional handcrafted features like review length or punctuation count were added, keeping the feature space purely text-based.

5. Model Development

The model we used is Logistic Regression, trained on the TF-IDF feature matrix. The input to the model is a sparse matrix of TF-IDF weights, one row per review, one column per vocabulary token, and the output is a binary prediction: 1 for positive sentiment and 0 for negative sentiment.

The data was split 80/20 into training and test sets using a fixed random seed (42) for reproducibility. The logistic regression was trained with a maximum of 1,000 iterations to ensure convergence.

The model was evaluated on the held-out test set using four standard classification metrics:

Accuracy: 0.78

Precision: 0.84

Recall: 0.71

F1-Score: 0.77

The overall accuracy of 78% is solid for a text classification task of this kind. The high precision (0.84) means that when the model predicts a review is positive, it is right about 84% of the time. The lower recall (0.71) tells us the model is more conservative, it misses some genuinely positive reviews rather than over-predicting positivity. In practical terms, this means the ranking system is somewhat conservative: restaurants need to earn positive sentiment predictions rather than getting the benefit of the doubt.

Given the binary nature of the labels (only 1s and 5s in the ratings), the task is well-defined and the model performs reliably. The F1-score of 0.77 reflects a reasonable balance between precision and recall.

6. Showcase

The trained logistic regression model and the fitted TF-IDF vectorizer are serialized together into a file called `model_bundle.pkl` using Python's pickle module. This file is loaded by the Flask web application (`app.py`) at startup, so no training happens at inference time.

The web application exposes two main features. First, it computes a ranked list of all restaurants in the dataset by applying the following scoring formula to each restaurant:

$$\text{final_score} = (\text{average_rating} \times 0.6) + (\text{average_sentiment} \times 5 \times 0.4)$$

This formula weights the numerical rating at 60% and the model-predicted sentiment at 40%, combining both signals into a single score that drives the ranking. The sentiment component is scaled to the same 1-5 range as the rating before the weighted sum is computed.

Second, the application provides a text input field where a user can type any review and receive an instant prediction of whether it expresses positive or negative sentiment. This makes the underlying ML model directly accessible and gives users a way to experiment with the system.

The top-ranked restaurant in the dataset is Harbor House, with a final score of 3.076, followed by Riverside Grill (2.96) and The Garden Fork (2.92). The rankings reflect both the raw average rating and the proportion of reviews that the model classified as positive.

7. Discussion and Conclusions

The project successfully demonstrated how a relatively simple machine learning pipeline, TF-IDF vectorization combined with logistic regression, can be used to extract meaningful sentiment signals from text and incorporate them into a practical ranking system.

A few issues came up during the project worth noting. The dataset's binary rating distribution (only 1s and 5s) makes the classification task somewhat artificial, since real-world restaurant ratings follow a more gradual spread. In a real deployment, you would want a dataset with more varied ratings to test whether the sentiment labels derived from a threshold still hold up. The model also does not currently handle sarcasm, which is a known weakness of bag-of-words approaches.

Another limitation is that the scoring formula's weights (0.6 for rating, 0.4 for sentiment) were chosen based on intuition rather than systematic optimization. A more rigorous approach would tune these weights using a held-out validation set or user feedback.

On the positive side, the project gave us hands-on experience with the full machine learning workflow: data collection, preprocessing, feature engineering, model training, evaluation, and deployment. It also pushed us to think about how a model's output can be made meaningful to

end users through a well-designed interface, rather than just reporting a test accuracy and stopping there.

The biggest course concepts that came through in this project were text feature extraction, binary classification, and the importance of proper train/test splitting. We also came to appreciate that model deployment is its own challenge, getting a trained model into a running web app requires thinking about serialization, loading times, and user-facing error handling that are separate from the modeling work itself.

8. AI Usage

AI tools were used in a supporting role throughout this project. We used Claude primarily for two purposes: writing boilerplate code and debugging. For boilerplate, AI helped generate the initial Flask route structure and the HTML template skeleton, which saved time on parts of the project that were not the core ML focus.

For debugging, we described error messages we encountered during model serialization and Flask integration, and AI helped us identify the cause (incorrect pickle loading order) and fix it. We also used AI to double-check our TF-IDF setup and confirm that we were fitting the vectorizer only on training data.

The core machine learning decisions, the choice of logistic regression, the TF-IDF feature approach, the scoring formula, and the evaluation metrics were made by us based on what we learned in the course. AI was a tool for execution and debugging, not for the conceptual design of the system.